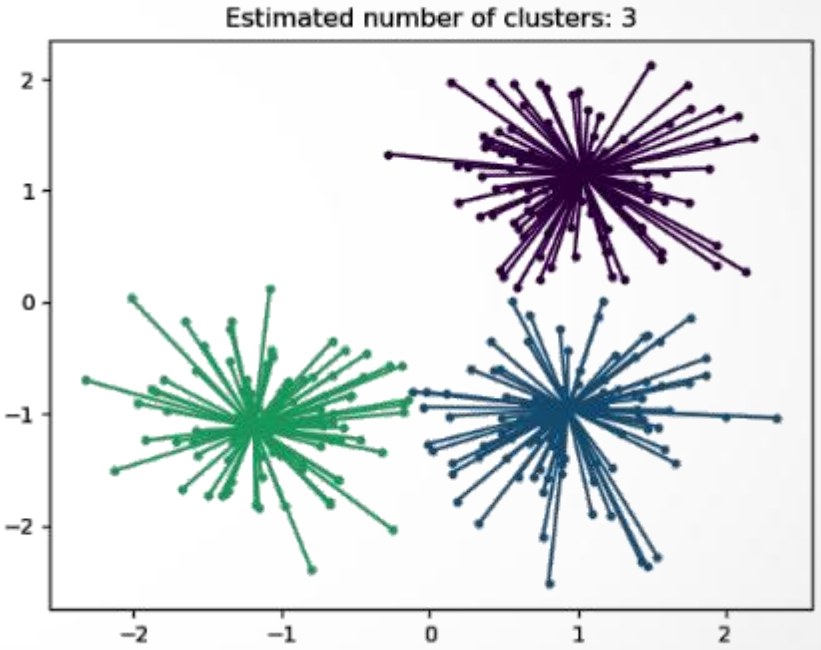
Clustering Algorithms

Affinity Propagation Clustering

Affinity Propagation is a clustering algorithm that identifies exemplars among data points and forms clusters based on message passing between data points.

The key idea behind Affinity Propagation is that each data point can both act as an exemplar (representative) of its own cluster and can also have a preference for being the exemplar of other data points. The algorithm seeks to find the exemplars that result in the highest total preference among all data points.



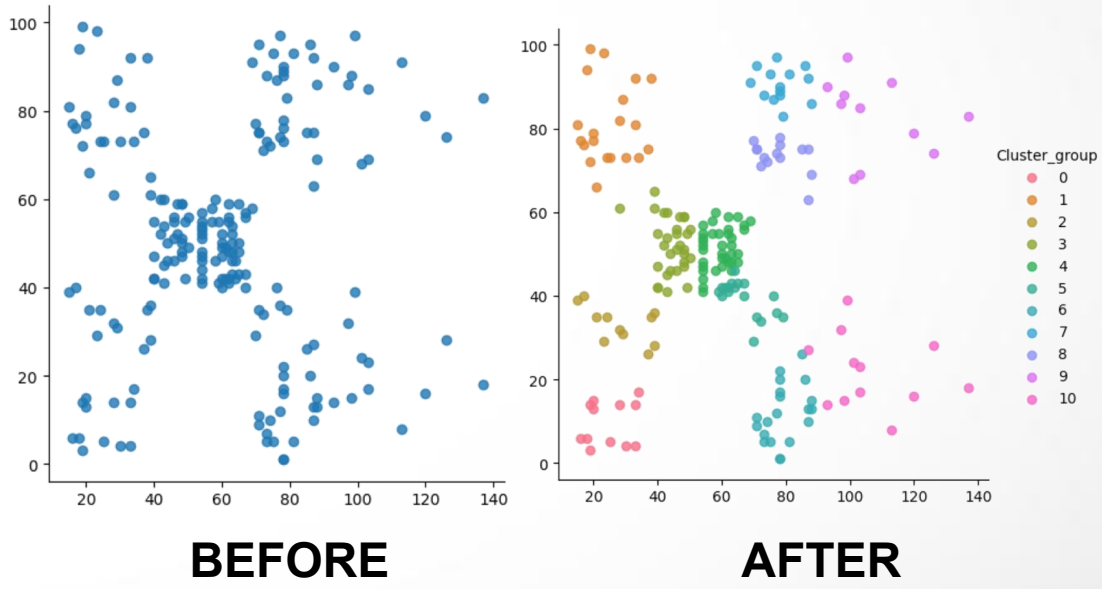
Affinity Propagation Clustering

Working Principles:

- It does not require the number of clusters to be specified.
- Each point sends and receives messages to evaluate its suitability as a cluster center (exemplar).
- Messages include “responsibility” and “availability” scores.

When to Use:

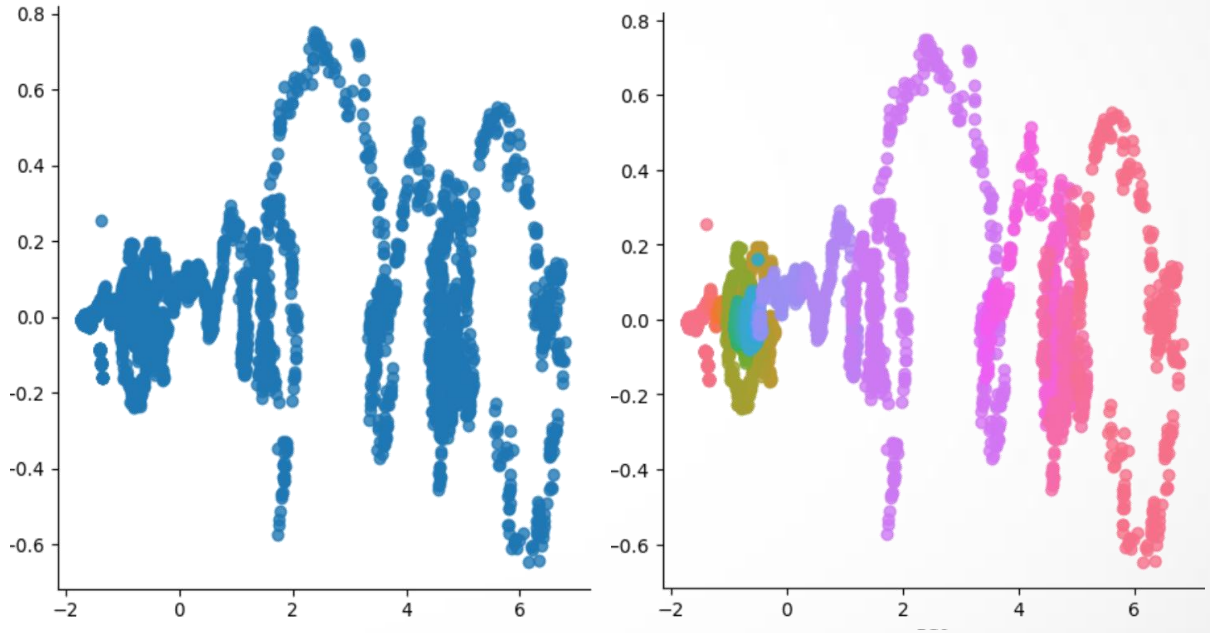
- When the number of clusters is unknown.
- When you want to automatically identify representative samples.



Affinity Propagation Clustering

Working Flow of Algorithm

1. Calculate similarity matrix.
2. Initialize messages.
3. Update responsibilities.
4. Update availabilities.
5. Identify exemplars.
6. Assign points to exemplars.
7. Repeat until convergence.



BEFORE AFTER

Affinity Propagation Clustering

```
from sklearn.cluster import AffinityPropagation
afp = AffinityPropagation(random_state=10)
y_afp = afp.fit_predict(X)
```

Advantages

- ☑ No need to specify number of clusters.
- ☑ Can find non-spherical clusters.
- ☑ Robust to noise.

Disadvantages

- ☒ Computationally intensive.
- ☒ Sensitive to input preference parameters.

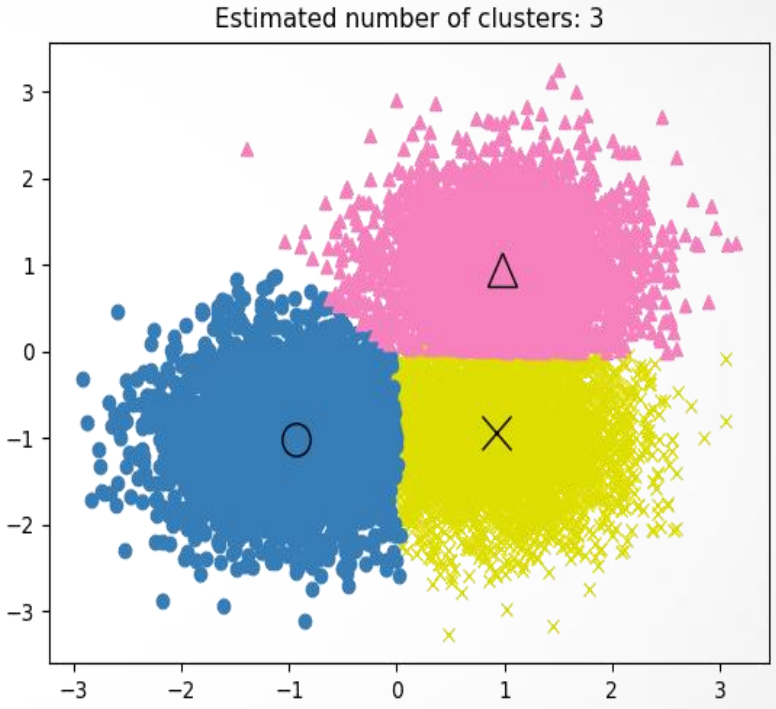
Applications Used

- ★ Face recognition.
- ★ Recommendation systems.

Mean Shift Clustering

MeanShift clustering aims to discover blobs in a smooth density of samples. It is a centroid based algorithm, which works by updating candidates for centroids to be the mean of the points within a given region. These candidates are then filtered in a post-processing stage to eliminate near-duplicates to form the final set of centroids.

The key idea behind this clustering technique is to find dense areas (modes) in the data by iteratively shifting points toward the mode.



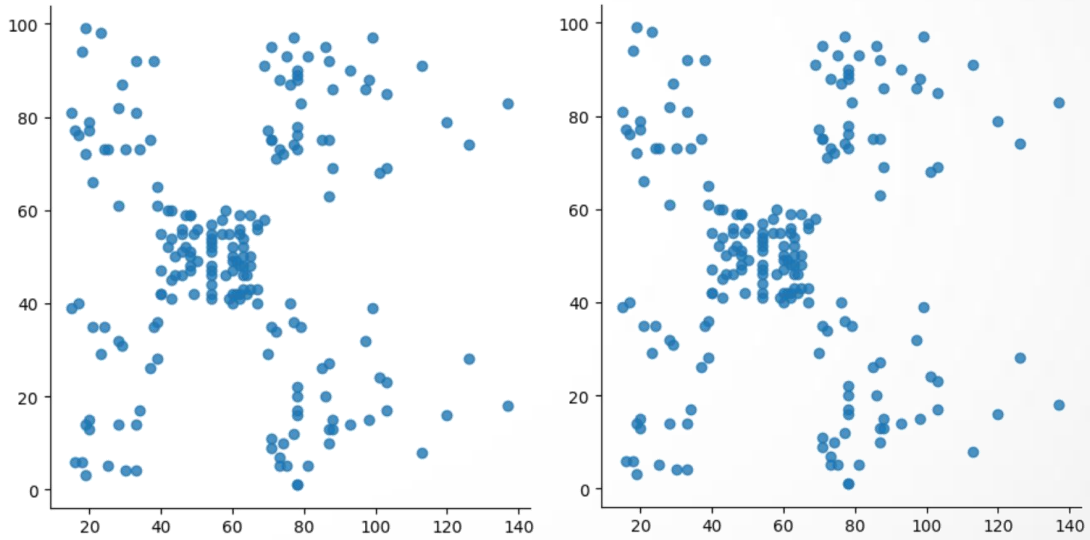
Mean Shift Clustering

Working Principles:

- Uses a sliding window to move towards regions with higher data density.
- Converges when points stop shifting.

When to Use:

- When the number of clusters is unknown.
- When clusters are not spherical.
- When working with continuous-valued features

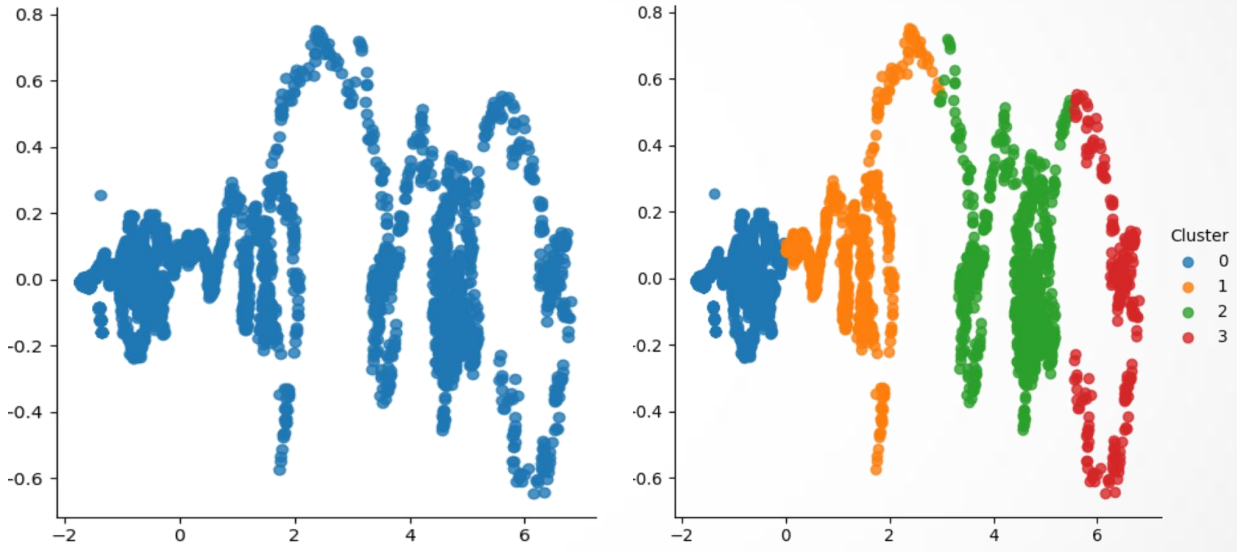


BEFORE **AFTER**

Mean Shift Clustering

Working Flow of Algorithm

1. Select bandwidth and kernel.
2. Initialize cluster centers.
3. Update cluster centers by shifting.
4. Check for convergence.
5. Merge nearby centers.



BEFORE **AFTER**

Mean Shift Clustering

```
from sklearn.cluster import MeanShift
from sklearn.cluster import estimate_bandwidth

bandwidth = int(estimate_bandwidth(X, quantile=0.5))
mnshift = MeanShift(bandwidth=bandwidth).fit_predict(X)
```

Advantages

- ☑ No need to specify number of clusters.
- ☑ Can find arbitrarily shaped clusters.
- ☑ Good for data smoothing.

Disadvantages

- ☒ Computationally expensive.
- ☒ Choice of bandwidth critical.

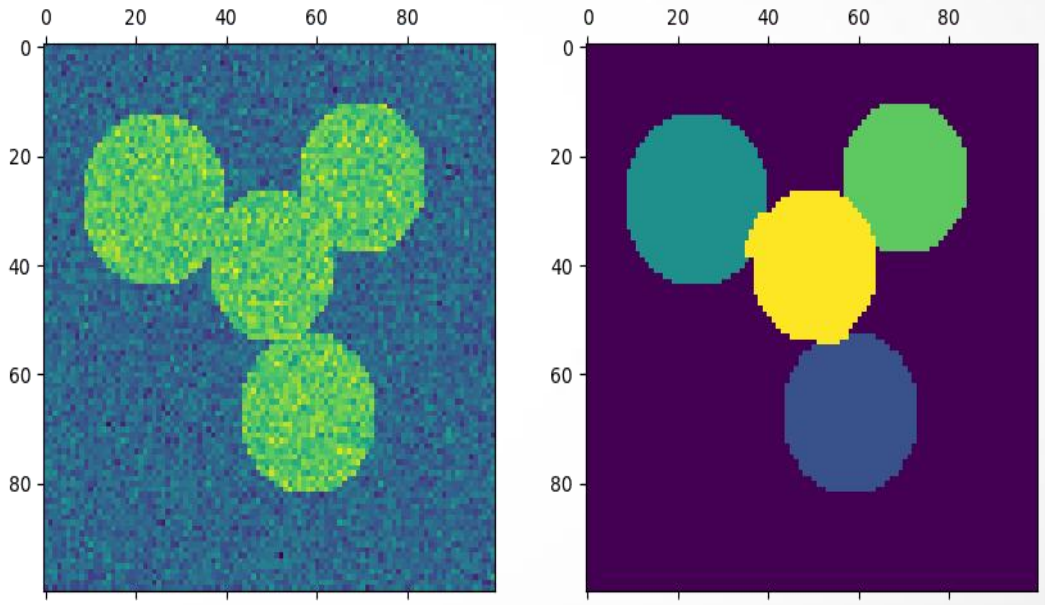
Applications Used

- ★ Image segmentation.
- ★ Object tracking.

Spectral Clustering

SpectralClustering performs a low-dimension embedding of the affinity matrix between samples, followed by clustering of the components of the eigenvectors in the low dimensional space.

The key idea behind this clustering technique is it uses the eigenvalues of a similarity matrix to perform dimensionality reduction before applying clustering techniques, like K-Means



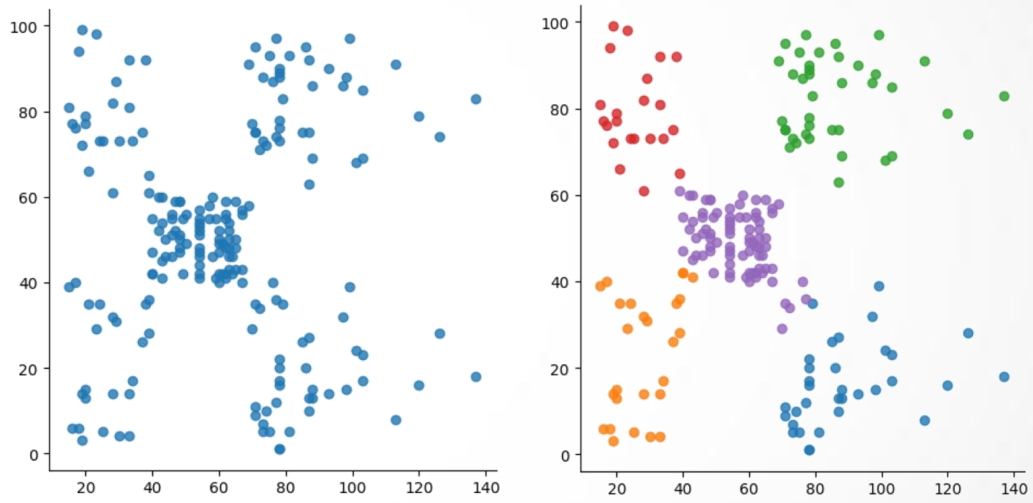
Spectral Clustering

Working Principles:

- Constructs a graph from data points.
- Applies graph partitioning methods.
- Eigenvectors of Laplacian matrix define the embedding.

When to Use:

- When dealing with non-convex or non-linear data.
- When the dataset has a graph-like structure.

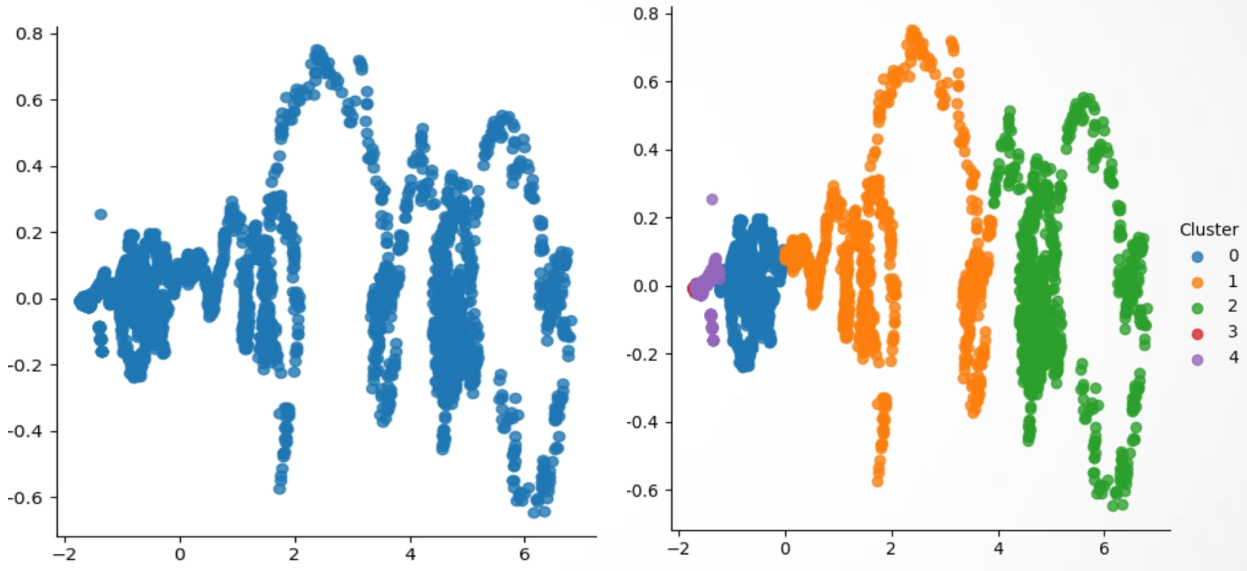


BEFORE **AFTER**

Spectral Clustering

Working Flow of Algorithm

1. Compute similarity matrix.
2. Construct Laplacian.
3. Calculate eigenvectors.
4. Form feature matrix.
5. Apply k-means.



BEFORE **AFTER**

Spectral Clustering

```
from sklearn.cluster import SpectralClustering

spclust = SpectralClustering(n_clusters=5, affinity='nearest_neighbors', random_state=10).fit_predict(X)
```

Advantages

- ☑ Captures complex relationships.
- ☑ Works well for graph-based data.
- ☑ Effective for non-convex clusters.

Disadvantages

- ☒ Computationally expensive.
- ☒ Requires number of clusters.

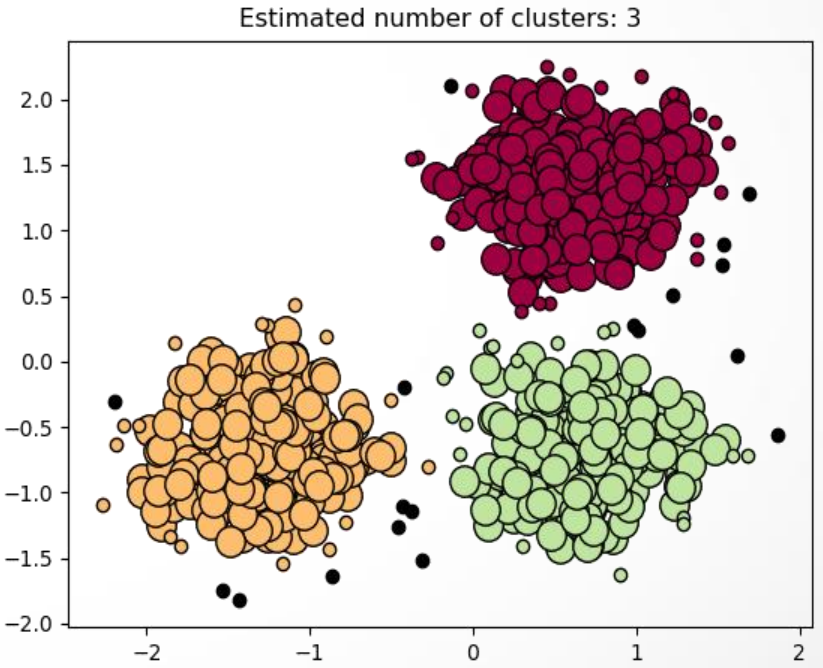
Applications Used

- ★ Image segmentation.
- ★ Clustering non-convex shapes.

DBSCAN Clustering

The **D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oisealgorithm (DBSCAN) views clusters as areas of high density separated by areas of low density. Due to this rather generic view, clusters found by DBSCAN can be any shape, as opposed to k-means which assumes that clusters are convex shaped. The central component to the DBSCAN is the concept of core samples, which are samples that are in areas of high density.

The key idea behind this clustering technique is that it groups together closely packed points and marks points in low-density areas as outliers.



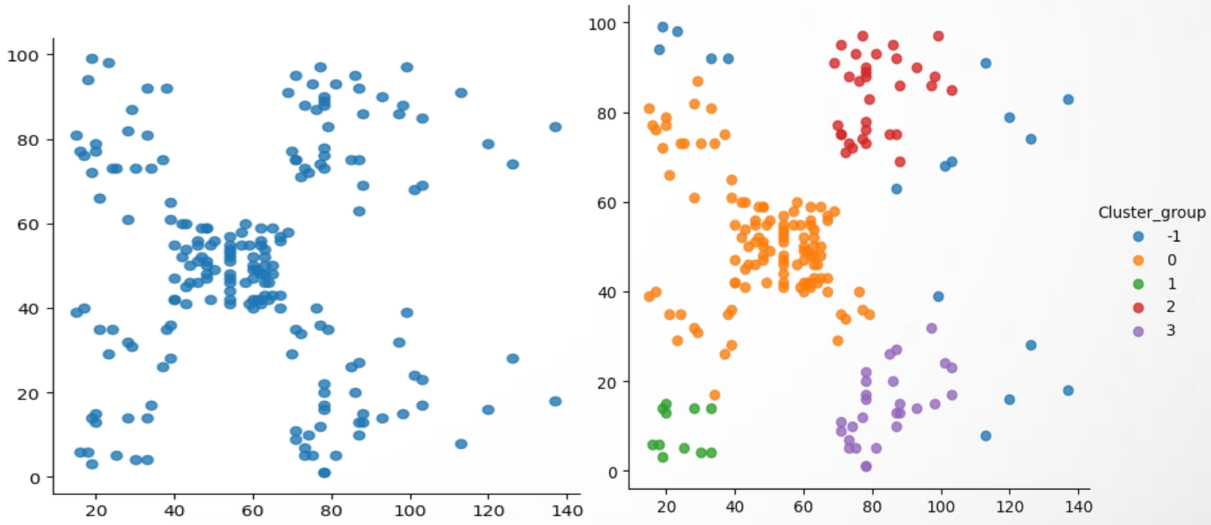
DBSCAN Clustering

Working Principles:

- Classify points as core, border, or noise.
- Expand clusters from core points.
- Ignore noise.

When to Use:

- When clusters have arbitrary shape.
- When data has noise or outliers.

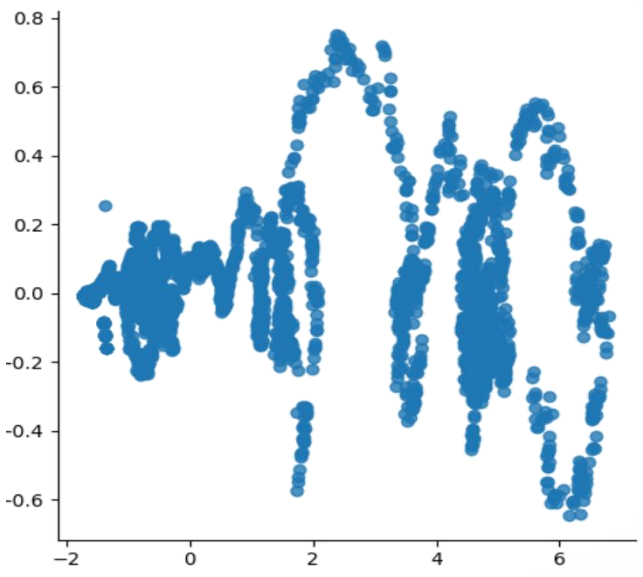


BEFORE **AFTER**

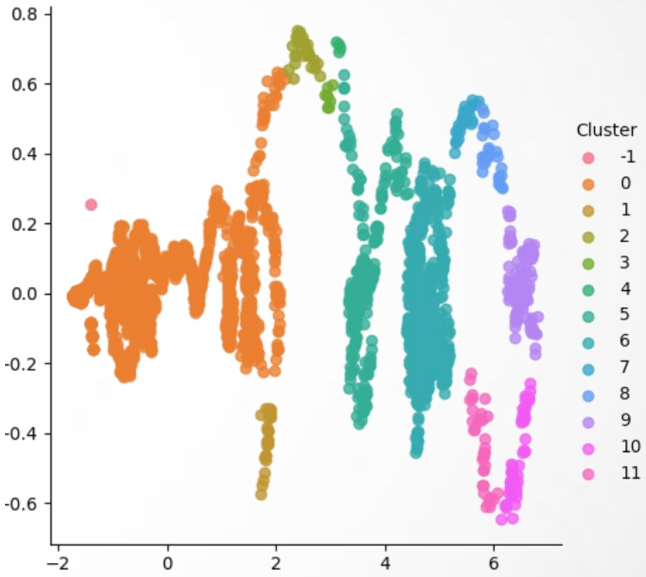
DBSCAN Clustering

Working Flow of Algorithm

1. Select eps and minPts.
2. Label core/border/noise points.
3. Expand clusters from core points.
4. Merge density-reachable points.



BEFORE



AFTER

DBSCAN Clustering

```
from sklearn.cluster import DBSCAN

dbscan_clust = DBSCAN(eps=10, min_samples=4)
y_dbscan_clust = dbscan_clust.fit_predict(X)
```

Advantages

- ☑ Detects arbitrarily shaped clusters.
- ☑ Identifies noise and outliers.
- ☑ No need for K.

Disadvantages

- ☒ Difficult to select eps and minPts.
- ☒ Struggles with varying densities.

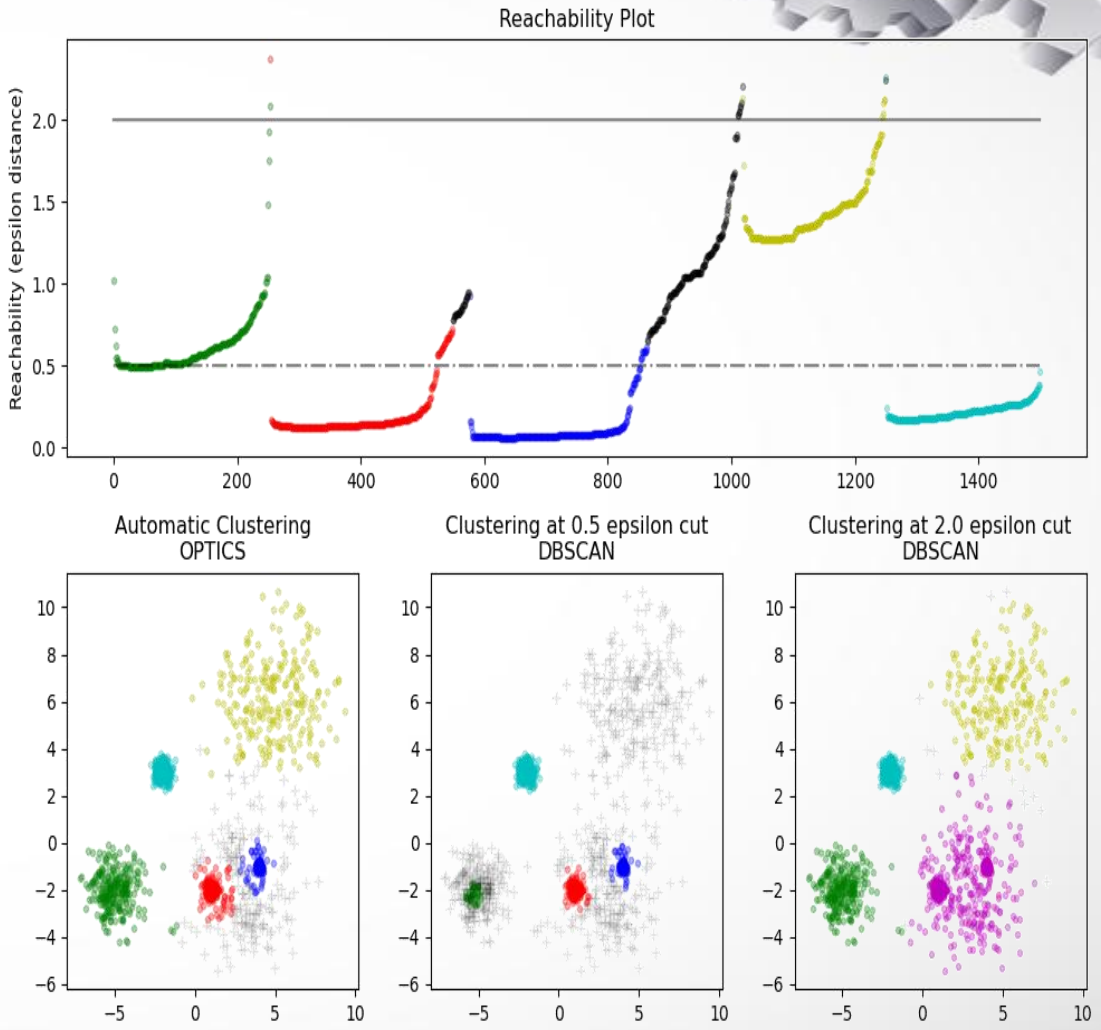
Applications Used

- ★ Fraud detection.
- ★ Geospatial data clustering.

OPTICS Clustering

The **O**rdering **P**oints **T**o **I**dentify the **C**lustering **S**tructure (OPTICS) algorithm shares many similarities with the DBSCAN algorithm, and can be considered a generalization of DBSCAN that relaxes the eps requirement from a single value to a value range.

The key difference between DBSCAN and OPTICS is that the OPTICS algorithm builds a reachability graph, which assigns each sample both a reachability_ distance, and a spot within the cluster ordering_ attribute; these two attributes are assigned when the model is fitted, and are used to determine cluster membership.



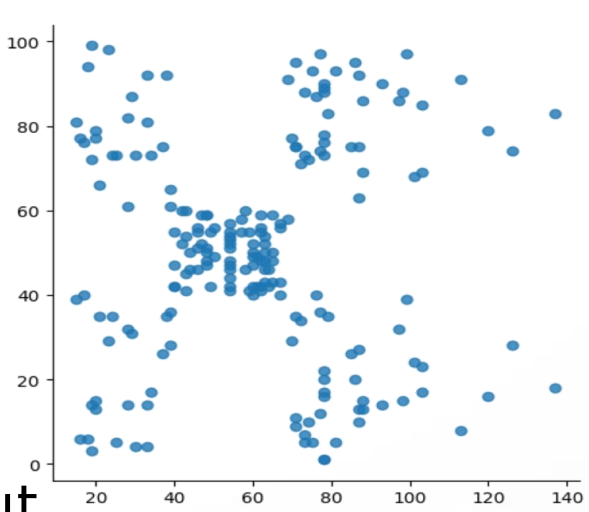
OPTICS Clustering

Working Principles:

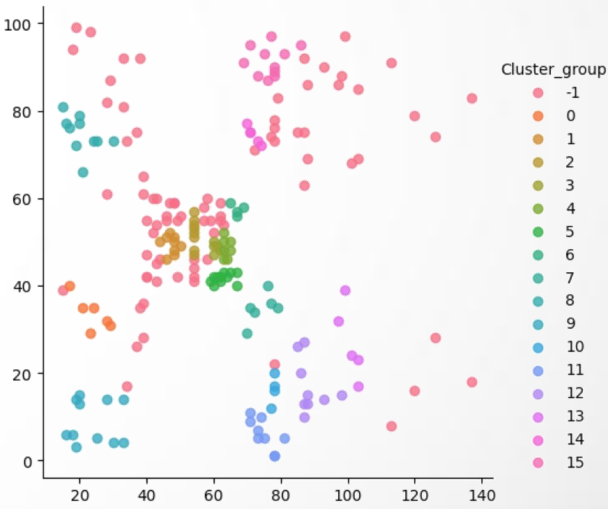
- Instead of assigning cluster labels directly, it produces a reachability plot
- Classify points as core, border, or noise.
- Expand clusters from core points.
- Ignore noise.

When to Use:

- When data has variable density without strict eps.
- When data has noise or outliers.



BEFORE

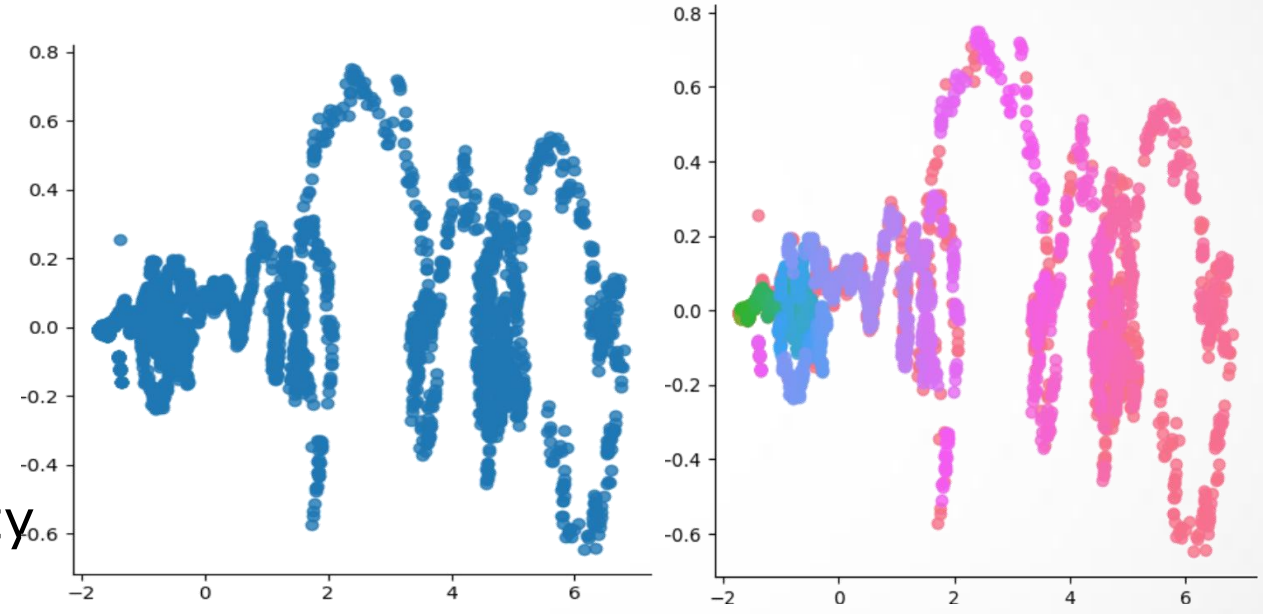


AFTER

OPTICS Clustering

Working Flow of Algorithm

1. Set parameters.
2. Initialize reachability plot.
3. Traverse points to build ordering.
4. Extract clusters from reachability plot.



BEFORE **AFTER**

OPTICS Clustering

```
from sklearn.cluster import OPTICS

optics_clust = OPTICS( min_samples=4)
Y_optics_clust = optics.fit_predict(X)
```

Advantages

- ☑ Handles varying densities.
- ☑ Identifies noise and outliers.
- ☑ No strict parameter tuning.

Disadvantages

- ☒ Slower than DBSCAN.
- ☒ More complex to interpret.

Applications Used

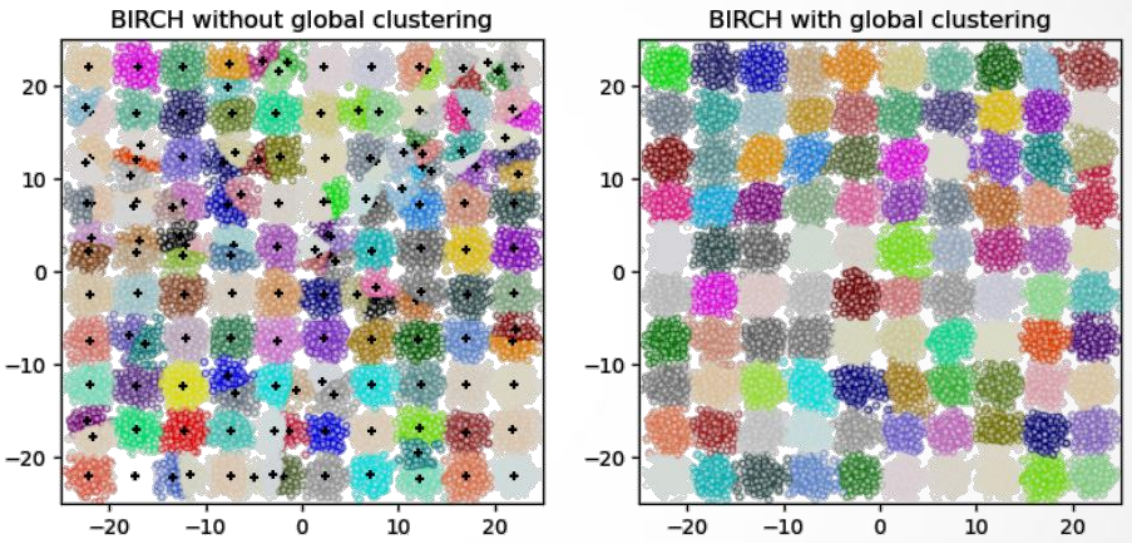
- ★ Astronomy.
- ★ Earthquake data.

BIRCH Clustering

The **B**alanced **I**terative **R**educing and **C**lustering using **H**ierarchies (BIRCH) builds a tree called the Clustering Feature Tree (CFT) for the given data. The data is essentially lossy compressed to a set of Clustering Feature nodes (CF Nodes).

The CF Nodes have a number of subclusters called Clustering Feature subclusters (CF Subclusters) and these CF Subclusters located in the non-terminal CF Nodes can have CF Nodes as children.

This algorithm is designed for large datasets and incrementally builds a tree structure (CF Tree) for clustering.



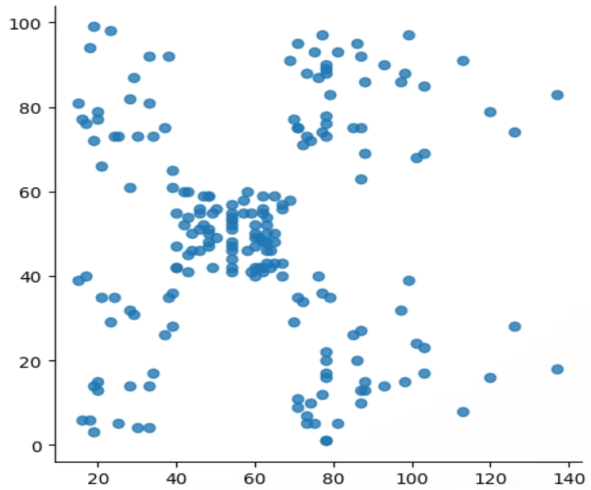
BIRCH Clustering

Working Principles:

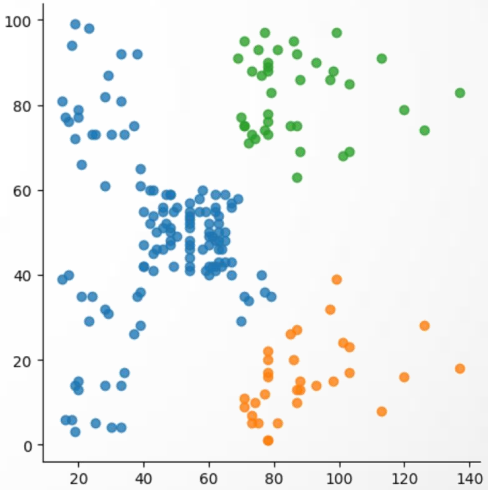
- Uses a compact representation of data in the form of Clustering Features (CF).

When to Use:

- For large datasets
- When memory efficiency is crucial.



BEFORE

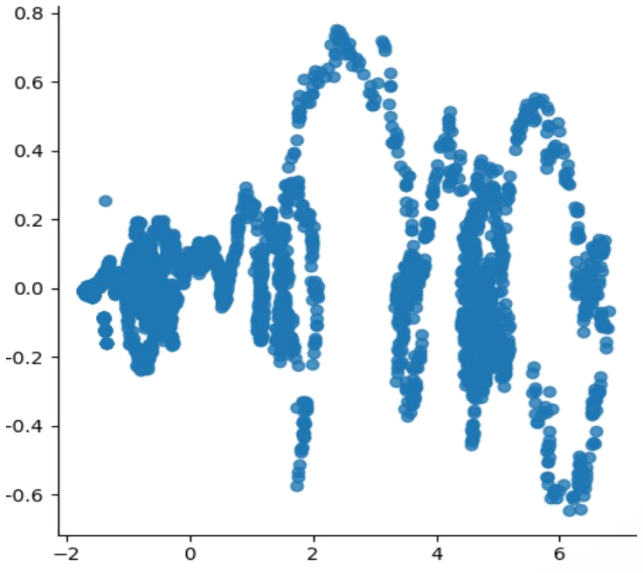


AFTER

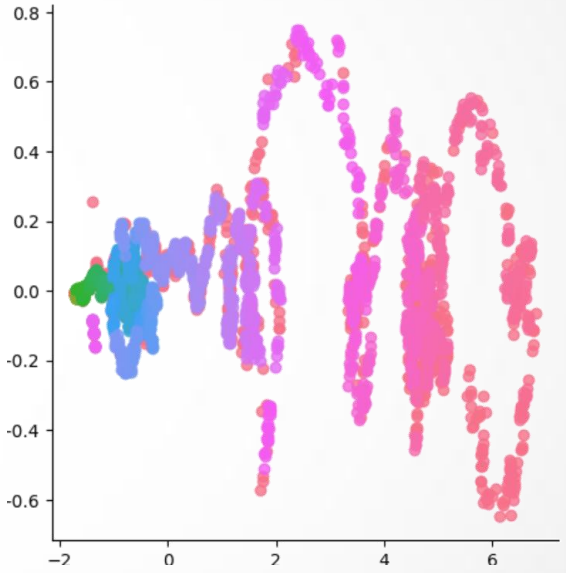
BIRCH Clustering

Working Flow of Algorithm

1. Set branching factor and threshold
2. Build CF Tree.
3. Rebuild tree if necessary.
4. Apply clustering on CF leaves.



BEFORE



AFTER

BIRCH Clustering

```
from sklearn.cluster import Birch

birch_clust = Birch()
y_birch_clust = birch_clust.fit_predict(X)
```

Advantages

- ☑ Efficient on large datasets.
- ☑ Incremental and online learning.
- ☑ No strict parameter tuning.

Disadvantages

- ☒ Assumes spherical clusters.
- ☒ Poor with non-uniform cluster sizes.

Applications Used

- ★ Large-scale pattern recognition.
- ★ Transaction clustering.